Group Project: Sharing Ride Application

Class: CIS344

Prof: Yanilda

Title: **Sharing Ride Application**

Ibrahim Sallieu Korgay

Kadidia Ouedraogo

Kadiatou Diallo

3/29/2026.

COMMUNICATION TOOL WE USE FOR THIS PROJECT ARE ZOOM AND WHATSAAP.

Subtitle: Built with PHP, MySQL, and JavaScript.

Project Plan, Architecture & Core System Features Report by Kadiatou Diallo

This is a comprehensive project that demonstrates a full-stack web application from concept to completion. We will explore its architecture, features, and the key technologies that power it.

1. Project Plan & Architecture

2. Core System Features

3. Database Design & SQL Usage

4. Backend Logic & Transactions

5. Security Best Practices

6. Frontend Design & User Flow

7. Structure of Code & Documentation

We will start with the high-level project plan and architecture. Then, we dive into the core features and the database that supports them. We will pay special attention to the backend logic, including the use of SQL transactions and key security measures. After that, the frontend design, and provide a brief live demonstration if needed.

Project Plan: Scope & Objectives

Scope:

The scope of this application was to create a platform that connects passengers with drivers, much like popular services such as Uber or Lyft. The system is designed to solve the problem of finding reliable, on-demand transportation by centralizing booking, tracking, and payment.

Objectives:

Our key objectives were to build a system that allows for secure user management, enables riders to book rides effortlessly, lets drivers manage their availability, automatically matches riders with drivers, and maintains a complete history of all trips and payments, all while ensuring the integrity of our data.

System Architecture

The application is built on three architectures, which are the standard for scalable web applications.

- The **Frontend** is what the user sees and interacts with. It's built with HTML, CSS, and JavaScript.

- The **Backend** is the brain of the system. It's powered by PHP and handles all the business logic, from user authentication to ride booking. It exposes a JSON-based API that the frontend communicates with.
- The **Database** is responsible for storing and managing all our data. The front-end and backend communicate asynchronously. JavaScript sends requests to the PHP API, which then queries the database and returns a JSON response to the client.

System Features: Core Functionality

The application supports two user roles: riders and drivers, each with a distinct registration and login process. Security is paramount, so all passwords are securely hashed using the BCRYPT algorithm.

- **Driver Availability:** A key feature for drivers is the ability to manage their work status. They can set themselves as available to receive ride requests, or unavailable when they are offline. The system automatically sets them to busy when they accept a ride.

- **Ride Booking:** the booking process is simple. They enter a pickup and destination, and our system finds and assigns the first available driver, creating a new trip record.

<u>Database Report by Kadidia Ouedraogo</u>

Database Design

The foundation of our application is a well-designed relational database. **Table Description: users:** stores general account details such as Name, email, password, and role (rider or driver).

riders: Contains rider-specific detail and links to the Users table. **drivers:** Stores driver information including license number, and availability status and rating. **vehicles:** Stores vehicle details linked to drivers.

bookings: Stores ride requests including pickup, drop-off location, estimated fare and booking status.

trips: Stores actual trip details such as start time, end time, distance and total fare.

payments: stores payment information such as amount, method and payment status.

Relationships between Tables

-One user can either be a rider or a driver

-One rider can create multiple bookings

-One driver can handle multiple bookings

-Each booking generates one trip

-Each trip has one payment.

Primary keys and Foreign Keys

We used **Primary keys and foreign keys** to maintain relationships and make sure data integrity in the database has been established.

Primary keys: We used primary keys to identifies each record in a table. For instance, id in the users, riders, drivers, trips, and payments tables ensure that each record is unique.

Foreign Key:

We used a foreign key to link one table to another. For example:

rider_id in bookings links to the riders table.

rriver_id links to the drivers table

trip_id in payment links to the trips

table.

Theses relation ensures that all data remains consistent and properly connected across the system

These tables are linked using foreign keys to maintain referential integrity. A rider and a driver are both linked to a trip, and each trip is linked to a payment. This normalized structure prevents data redundancy and ensures consistency.

SQL JOINS

we rely heavily on SQL JOINs. Instead of running separate queries for trips, then drivers, then payments, we combine them into a single, optimized query. This is far more performant than handling the logic in our PHP code.

To show a rider their trip history, we join the Trips table with the Drivers table to get the driver's name. with the Payments table to get the payment status.

SQL joins are used to combine data from multiple tables in sharing_ride_systems. Since the database is normalized, information such as riders, drivers, bookings, trips and payments is stored on separate tables. Joins allow the application to retrieve complete and meaningful data in single query. For example, to display trip history, the system needs to show the rider's name, driver's name, pickup and drop-off location, fare and payment status. This information comes from different tables and is combined using SQL join.

Types of Joins query Used:

INNER JOIN:

Inner Join returns only records that have matching values in both tables. It is used when complete and connected data is required, such as displaying completed trip details

LEFT JOIN:

Left Join returns all records from one table even if there is no matching data in the related data table. It is useful for showing incomplete data, such as drivers without vehicle or booking without payment.

```sql
CREATE TABLE vehicles (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    driver_id INT UNSIGNED NOT NULL,
    vehicle_type VARCHAR(50) NOT NULL,
    brand VARCHAR(50) NOT NULL,
    model VARCHAR(50) NOT NULL,
    color VARCHAR(30) NOT NULL,
    plate_number VARCHAR(20) NOT NULL UNIQUE,
    seat_capacity TINYINT UNSIGNED NOT NULL DEFAULT 4,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_vehicles_driver FOREIGN KEY (driver_id) REFERENCES drivers(id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ;
```

```sql
CREATE TABLE bookings (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    rider_id INT UNSIGNED NOT NULL,
    driver_id INT UNSIGNED DEFAULT NULL,
    pickup_location VARCHAR(255) NOT NULL,
    dropoff_location VARCHAR(255) NOT NULL,
    estimated_distance_km DECIMAL(8,2) NOT NULL,
    estimated_duration_minutes INT UNSIGNED DEFAULT NULL,
    base_fare DECIMAL(10,2) NOT NULL DEFAULT 5.00,
    distance_rate DECIMAL(10,2) NOT NULL DEFAULT 2.00,
    estimated_fare DECIMAL(10,2) NOT NULL,
    booking_status ENUM('pending', 'accepted', 'cancelled', 'completed') NOT NULL DEFAULT 'pending',
    payment_method ENUM('cash', 'card', 'wallet') NOT NULL DEFAULT 'cash',
    requested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_bookings_rider FOREIGN KEY (rider_id) REFERENCES riders(id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_bookings_driver FOREIGN KEY (driver_id) REFERENCES drivers(id)
        ON DELETE SET NULL ON UPDATE CASCADE
) ;
```

```sql
CREATE TABLE trips (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    booking_id INT UNSIGNED NOT NULL UNIQUE,
    rider_id INT UNSIGNED NOT NULL,
    driver_id INT UNSIGNED NOT NULL,
    start_time DATETIME DEFAULT NULL,
    end_time DATETIME DEFAULT NULL,
    actual_distance_km DECIMAL(8,2) DEFAULT NULL,
    total_fare DECIMAL(10,2) NOT NULL,
    trip_status ENUM('ongoing', 'completed', 'cancelled') NOT NULL DEFAULT 'ongoing',
    notes TEXT DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_trips_booking FOREIGN KEY (booking_id) REFERENCES bookings(id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_trips_rider FOREIGN KEY (rider_id) REFERENCES riders(id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_trips_driver FOREIGN KEY (driver_id) REFERENCES drivers(id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ;
```

```sql
CREATE TABLE payments (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    trip_id INT UNSIGNED NOT NULL UNIQUE,
    amount DECIMAL(10,2) NOT NULL,
    payment_method ENUM('cash', 'card', 'wallet') NOT NULL,
    payment_status ENUM('pending', 'paid', 'failed', 'refunded') NOT NULL DEFAULT 'pending',
    transaction_reference VARCHAR(100) DEFAULT NULL UNIQUE,
    paid_at DATETIME DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_payments_trip FOREIGN KEY (trip_id) REFERENCES trips(id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ;
```

```sql
CREATE TABLE riders (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id INT UNSIGNED NOT NULL UNIQUE,
    default_pickup_location VARCHAR(255) DEFAULT NULL,
    preferred_payment_method ENUM('cash', 'card', 'wallet') DEFAULT 'cash',
    emergency_contact VARCHAR(20) DEFAULT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_riders_user FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE ON UPDATE CASCADE);
```

```sql
CREATE TABLE users (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE,
    phone VARCHAR(20) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('admin', 'rider', 'driver') NOT NULL,
    account_status ENUM('active', 'inactive', 'suspended') NOT NULL DEFAULT 'active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ;
```

```sql
CREATE TABLE drivers (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id INT UNSIGNED NOT NULL UNIQUE,
    license_number VARCHAR(50) NOT NULL UNIQUE,
    availability_status ENUM('available', 'busy', 'offline') NOT NULL DEFAULT 'offline',
    current_location VARCHAR(255) DEFAULT NULL,
    rating DECIMAL(3,2) DEFAULT 5.00,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_drivers_user FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ;
```

<u>System workflow Report by Ibrahim Sallieu Korgay</u>

Transactions for Data Integrity

One of the most critical backend processes is booking a ride. This single action requires three separate database operations: creating the trip, updating the driver's status, and creating a payment record. If any of these were to fail midway, our database would be in an inconsistent state. This makes the operation atomic either three queries succeed, and the changes are saved with COMMIT, if any one of them fails, the entire operation is undone with ROLLBACK. This guarantees our data remains consistent and reliable.

Security Enhancements

Security is a top priority throughout our development. We implemented several key best practices to protect our application and its users. First, to prevent SQL injections, every single database query that involves user data is executed using prepared statements. This ensures that user input is always treated as data, not as executable code.

We use the BCRYPT algorithm for password hashing. To prevent Cross-Site Scripting, or XSS, all user-provided input is sanitized before it is ever displayed on a page. Finally, we enhance session security by regenerating the session ID upon login, which protects against session fixation attacks.

Frontend Design & User Flow

We focused on creating a clean, intuitive, and responsive user interface. The design was built from scratch using modern CSS, including Flexbox and Grid, ensuring it works well on any device.

All user interactions, like logging in or booking a ride, are handled asynchronously by JavaScript using the Fetch API. This means the user gets instant feedback without having to wait

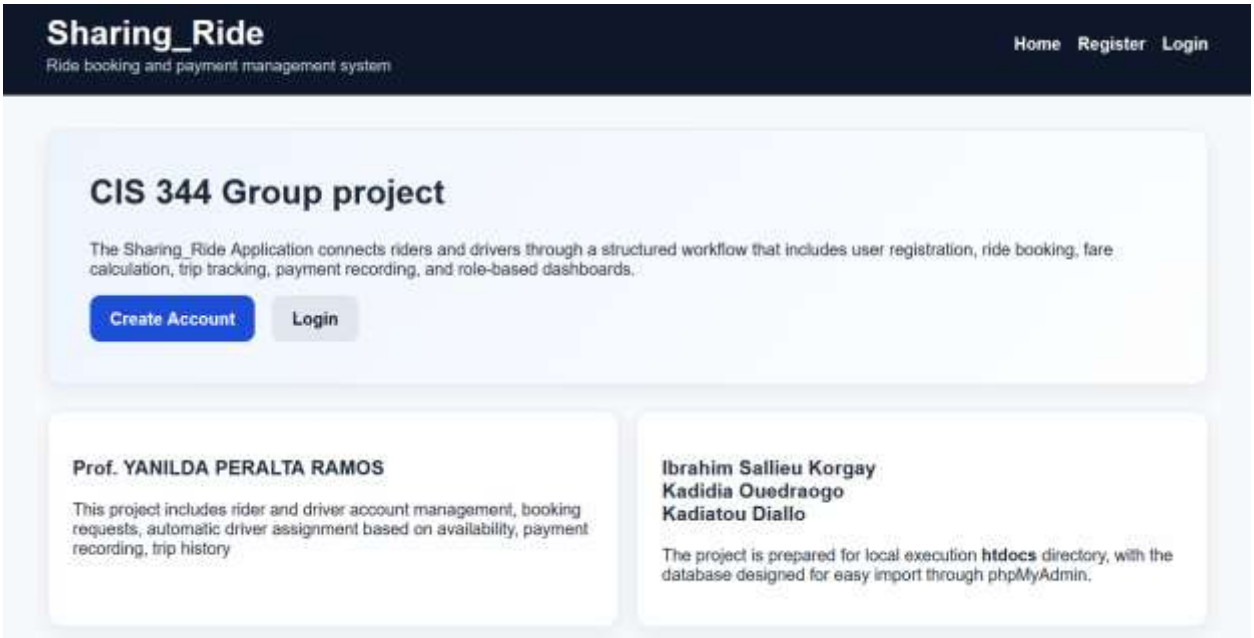
for a full page reload, creating a much smoother experience. The interface includes distinct dashboards for riders and drivers, a simple ride request form, and detailed history tables."

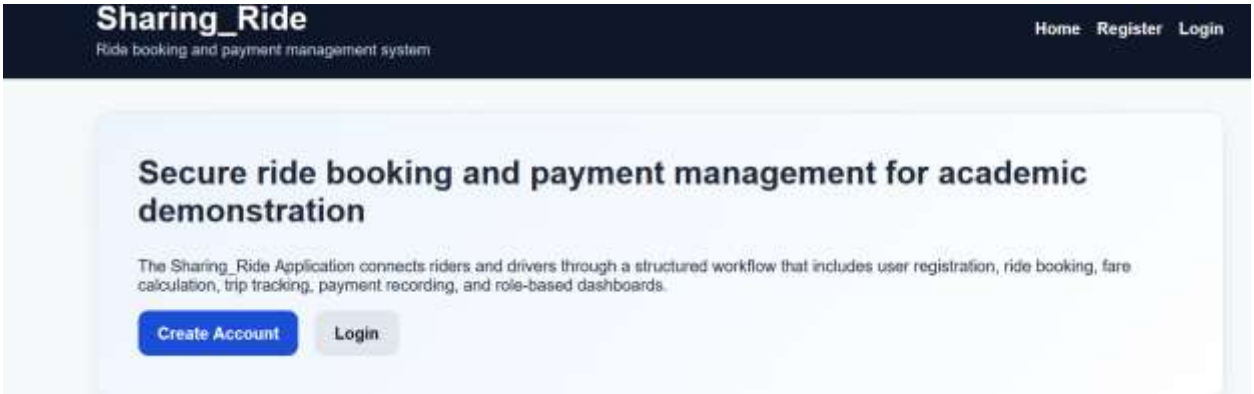
We followed a clear separation of concerns, keeping our PHP logic, HTML pages, CSS styles, and JavaScript in separate directories. This makes the codebase easy to navigate and extend.

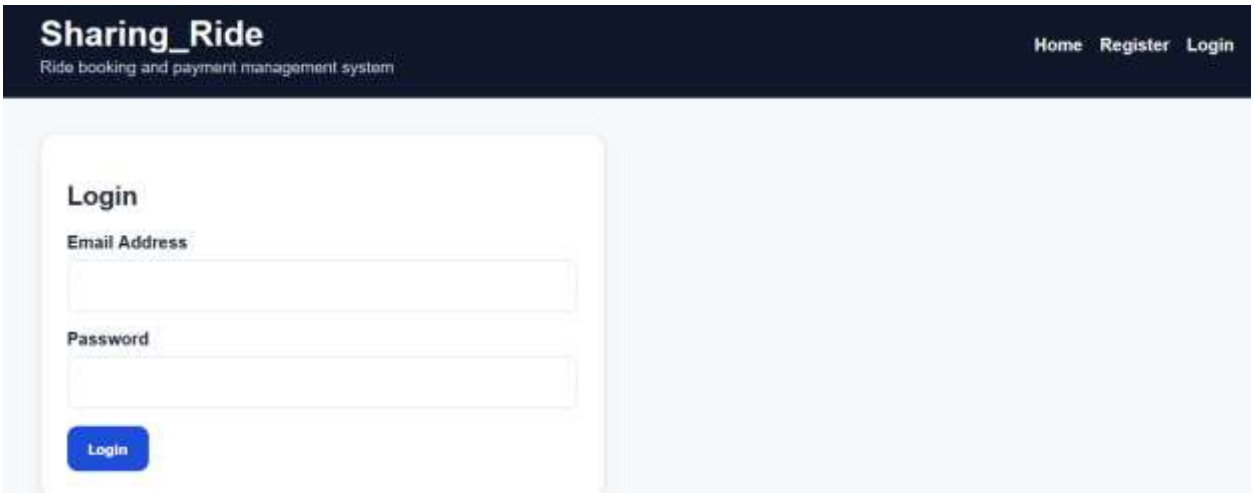
This project is also thoroughly documented. It includes a detailed academic write-up covering all the topics from this workflow as well as a practical README file.

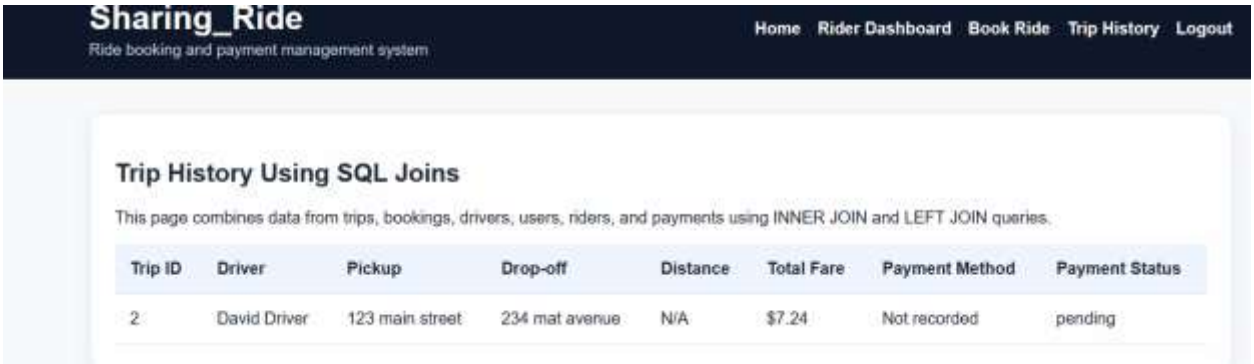
Conclusion

This project successfully demonstrates the development of a complete, full-stack web application. We have built a secure, feature-rich platform that applies best practices in database design, backend architecture, and security.

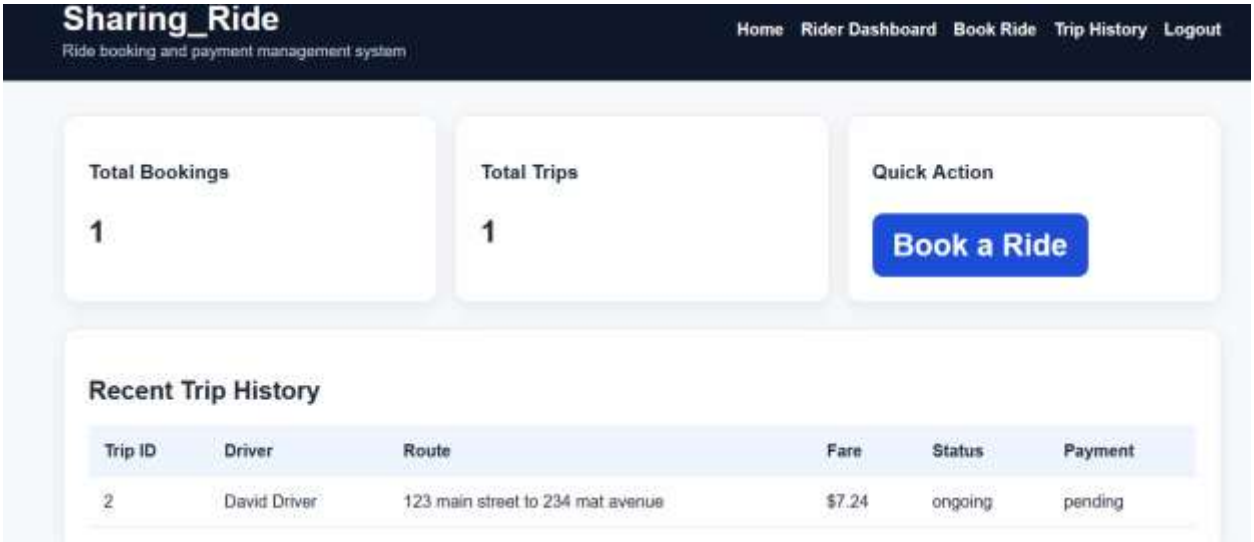


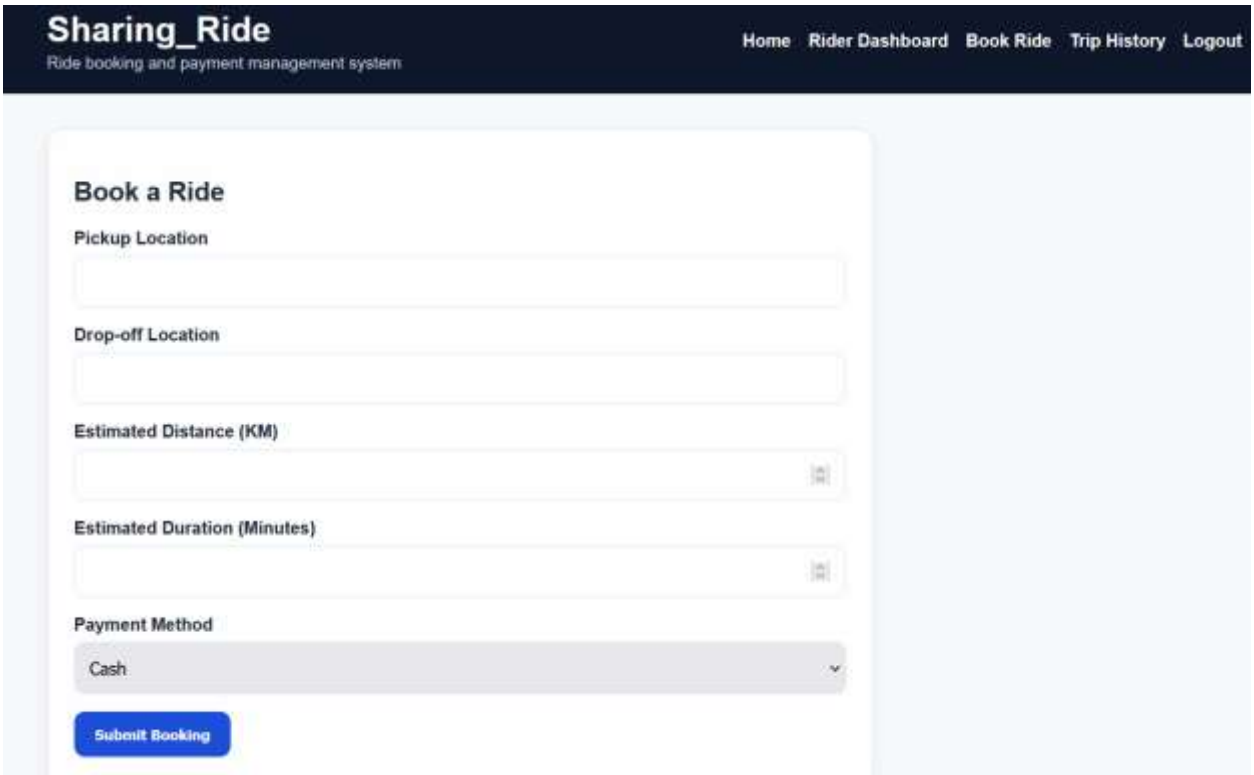






Trip ID	Driver	Pickup	Drop-off	Distance	Total Fare	Payment Method	Payment Status
2	David Driver	123 main street	234 mat avenue	N/A	$7.24	Not recorded	pending


Sharing_Ride
Ride booking and payment management system
Home
Rider Dashboard
Book Ride
Trip History
Logout
Total Bookings
1
Total Trips
1
Quick Action
Book a Ride
Recent Trip History
Trip ID
Driver
Route
Fare
Status
Payment
2
David Driver
123 main street to 234 mat avenue
$7.24
ongoing
pending


Sharing_Ride
Ride booking and payment management system
Home
Rider Dashboard
Book Ride
Trip History
Logout
Book a Ride
Pickup Location
Drop-off Location
Estimated Distance (KM)
Estimated Duration (Minutes)
Payment Method
Cash
Submit Booking

Sharing_Ride

Ride booking and payment management system

Create Account

Full Name

Email Address

Phone Number

Role

Rider

Password

Confirm Password